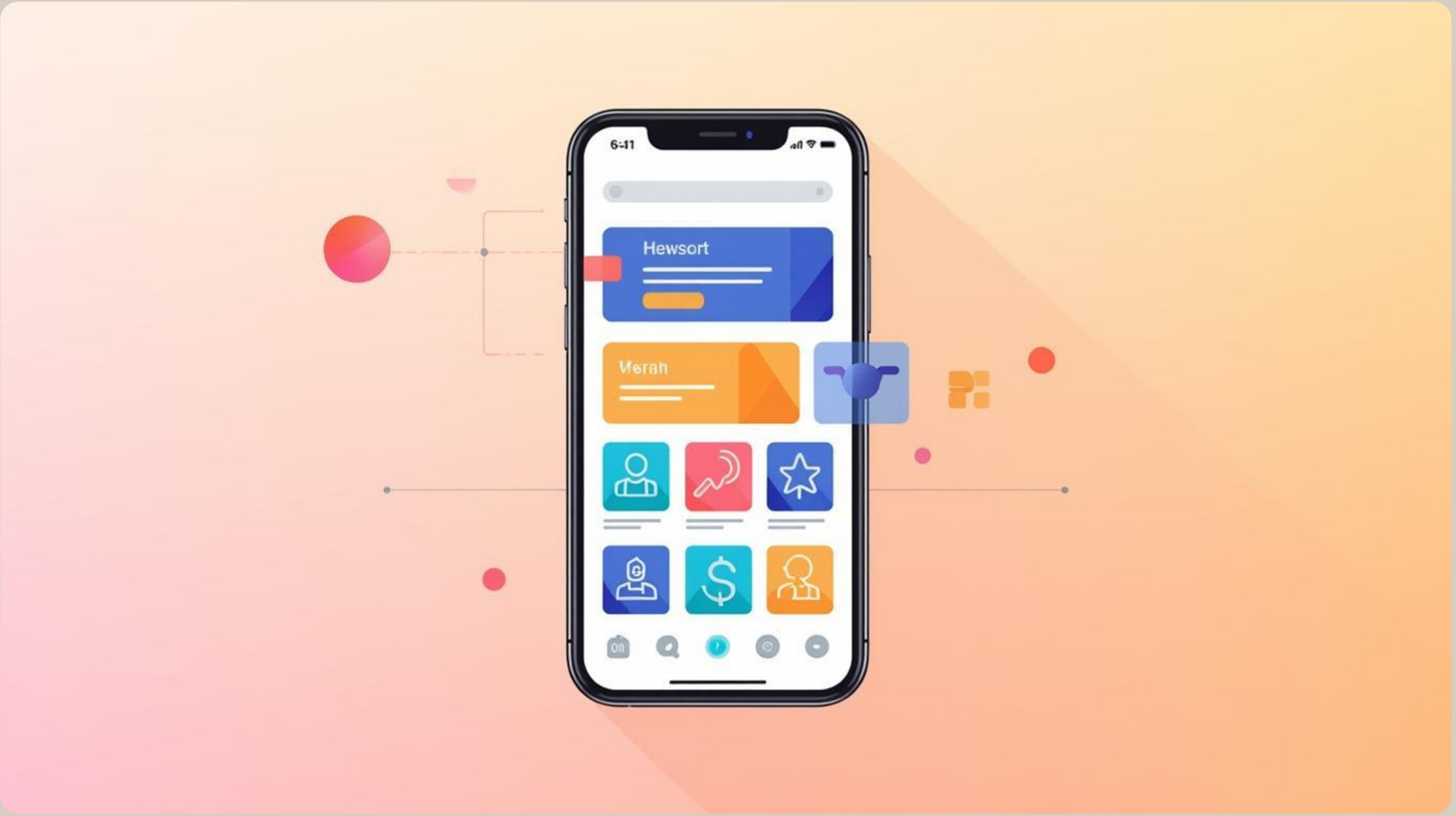
Responsivité Web

Dr Pascal Francois FAYE/ AgroTIC / USSEIN

Introduction à la Responsivité

- s’ajuster aux résolutions variées ;
- maintenir une lisibilité optimale ;
- adapter les images, grilles, menus et contenus ;
- garantir une ergonomie sans recourir au zoom.

12/01/2026

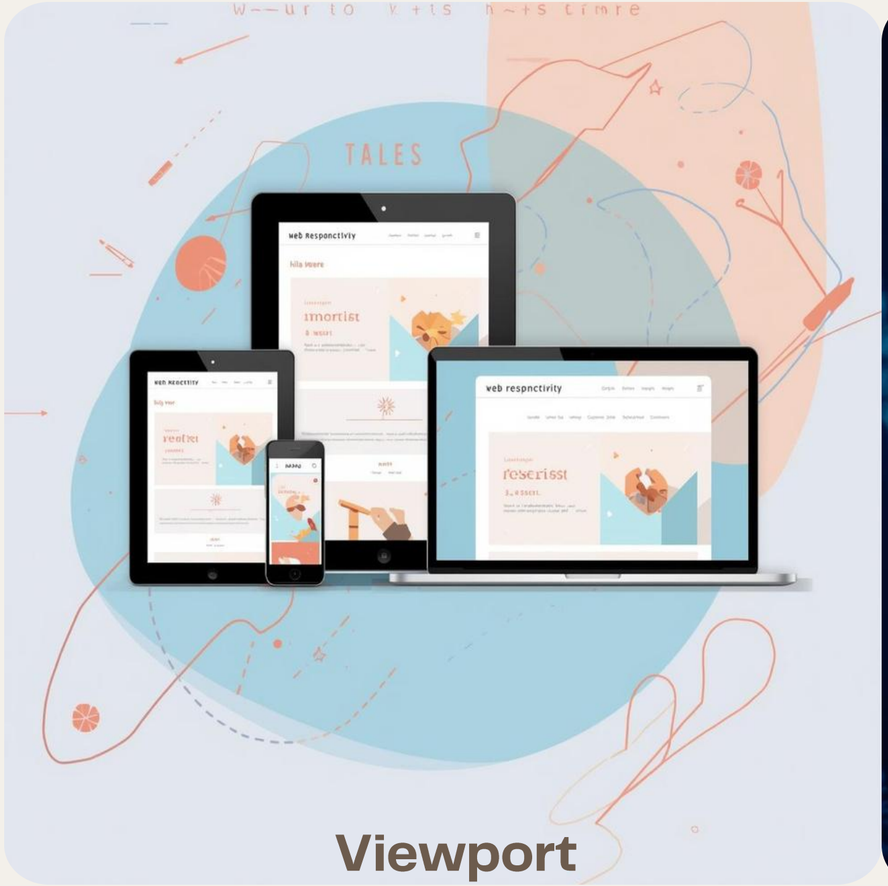


Site Responsif

Un site qui s'adapte à chaque taille d'écran.

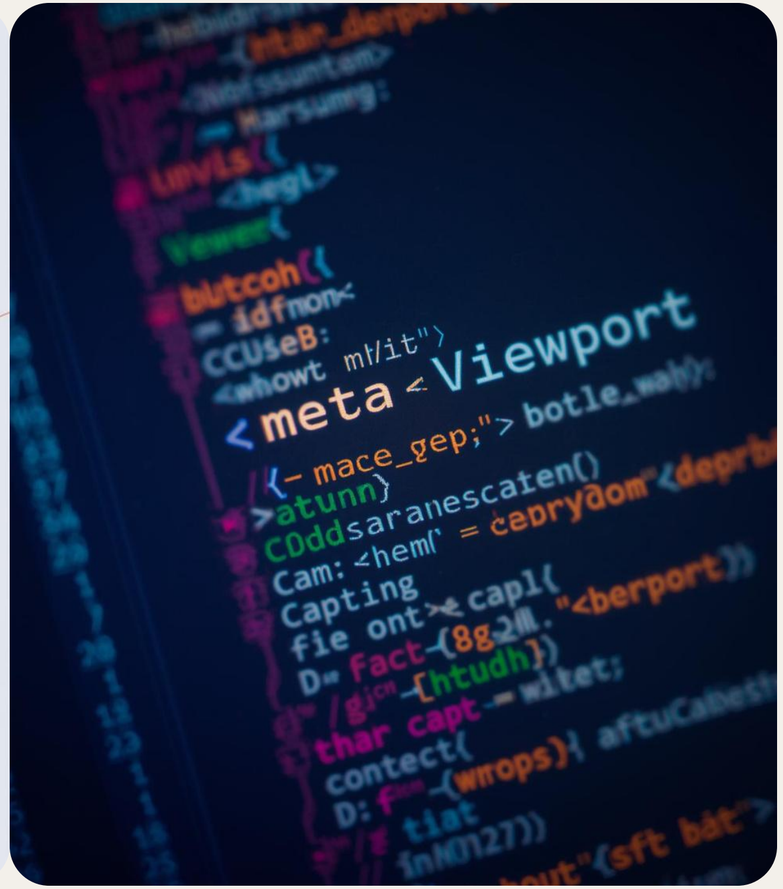
Importance du tag viewport

12/01/2026



Viewport

Essentiel pour adapter les pages web aux devices.



Code HTML

Intégration du code CSS contenant la partie qui gère la responsivité.

Pour activer la responsivité sur les mobiles :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Cette instruction dans la balise <head>, indique au navigateur d'afficher la page selon la largeur réelle du périphérique et d'établir un niveau de zoom initial de 100 % (initial-scale=1.0).

12/01/2026

Unités CSS flexibles

Ces unités permettent aux éléments de s'adapter fluidement.

rée une zone de contenu réactif : il occupe 90 % de la largeur de son parent mais cesse de croître à 1200px, garantissant ainsi que le contenu reste lisible sur les grands écrans tout en s'adaptant aux plus petits.

- **`%`** : tailles proportionnelles
- **`rem` / `em`** : tailles basées sur la police
- **`vw` / `vh`** : unités relatives à la fenêtre
- **`fr`** : fraction d'espace en CSS Grid

Exemple :

```
.container {
  width: 90%;
  max-width: 1200px;
}
```

Techniques de Layout Responsif

Les **media queries** permettent de modifier le style en fonction : - de la **largeur d'écran** - de l'**orientation** - de la **résolution**.

Syntaxe de base :

```
@media (max-width: 768px) {
  body { font-size: 14px; }
}
```

Exemples pratiques ➔

Adapter une grille :

```
@media (max-width: 768px) {
  .grid {
    grid-template-columns: 1fr;
  }
}
```

Réorganiser un menu :

```
@media (max-width: 600px) {
  nav ul {
    flex-direction: column;
  }
}
```

Principes de Flexbox

Alignement

Flexbox permet un **alignement dynamique** des éléments, facilitant leur positionnement vertical ou horizontal. Cette méthode garantit une disposition fluide, s'adaptant à la taille de l'écran et au contenu.

Réarrangement

Grâce à Flexbox, les éléments peuvent être **réorganisés facilement**. Sur un appareil mobile, les éléments initialement côte à côte peuvent être empilés verticalement, améliorant ainsi l'expérience utilisateur et la lisibilité.

Il crée un conteneur flexible où les éléments enfants sont distribués uniformément le long de l'axe principal (horizontalement par défaut), avec le premier élément aligné sur le bord de début et le dernier élément aligné sur le bord de fin.

```
.wrapper {
  display: flex;
  justify-content: space-between;
}

@media (max-width: 768px) {
  .wrapper {
    flex-direction: column;
    gap: 20px;
  }
}
```

flex-direction : colonne ; ➔ Empile les enfants directs du . wrapper verticalement au lieu de horizontalement (qui serait le défaut si display : flex était déjà appliqué à .wrapper.).
gap : 20px ; ➔ Ajoute 20 pixels d'espace entre les éléments enfants empilés

CSS Grid Layout

Grid permet de définir des grilles adaptatives.

```
div.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

Définit un conteneur de grille avec trois colonnes de largeur égale et un espace de 20 pixels entre les éléments de la grille. repeat(3, 1fr) crée une grille avec 3 colonnes de largeur égale (1/3).

```
@media (max-width: 900px) {
  div.grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (max-width: 600px) {
  div.grid {
    grid-template-columns: 1fr;
  }
}
```

12/01/2026

Responsive Design

Un flexible layout adapté à différentes tailles d'écran.

Structure modulaire

Organisé en sections améliorant la navigation et l'expérience utiliseur

12/01/2026

Navigation et Médias Responsifs

Adaptez votre site pour tous les appareils modernes.

22/01/2026

Responsivité des Images

Règle CSS pour les images adaptatives

La règle CSS ci-dessous garantit que les images s'ajustent parfaitement à leur conteneur, évitant le débordement tout en préservant leur rapport d'aspect.

```
img {
  max-width: 100%;
  height: auto;
}
```

Stratégies de Responsivité

Mobile-first

La stratégie mobile-first consiste à créer d'abord pour les petits écrans, en ajoutant des styles pour les écrans plus grands, garantissant ainsi une expérience utilisateur optimale sur tous les appareils.

12/01/2026

```
.grid { grid-template-columns: 1fr; }

@media (min-width: 600px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}
```

Desktop-first

La stratégie desktop-first commence par concevoir pour les grands écrans, puis réduit les styles pour les appareils plus petits, ce qui peut entraîner des problèmes d'affichage sur les mobiles.

```
.grid { grid-template-columns: repeat(3, 1fr); }

@media (max-width: 600px) {
  .grid { grid-template-columns: 1fr; }
}
```

Navigation Responsive

Un menu doit pouvoir se transformer.

Principe de base :

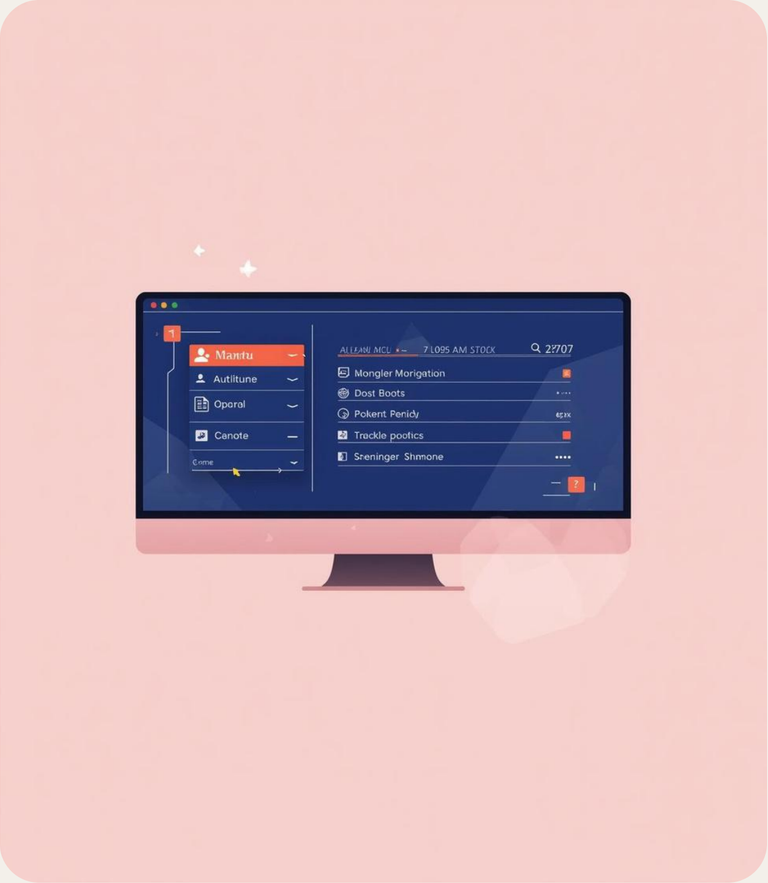
```
nav ul { display: flex; }

@media (max-width: 700px) {
  nav ul {
    flex-direction: column;
    display: none; /* masqué par défaut */
  }
}
```

12/01/2026

Hamburger Menu

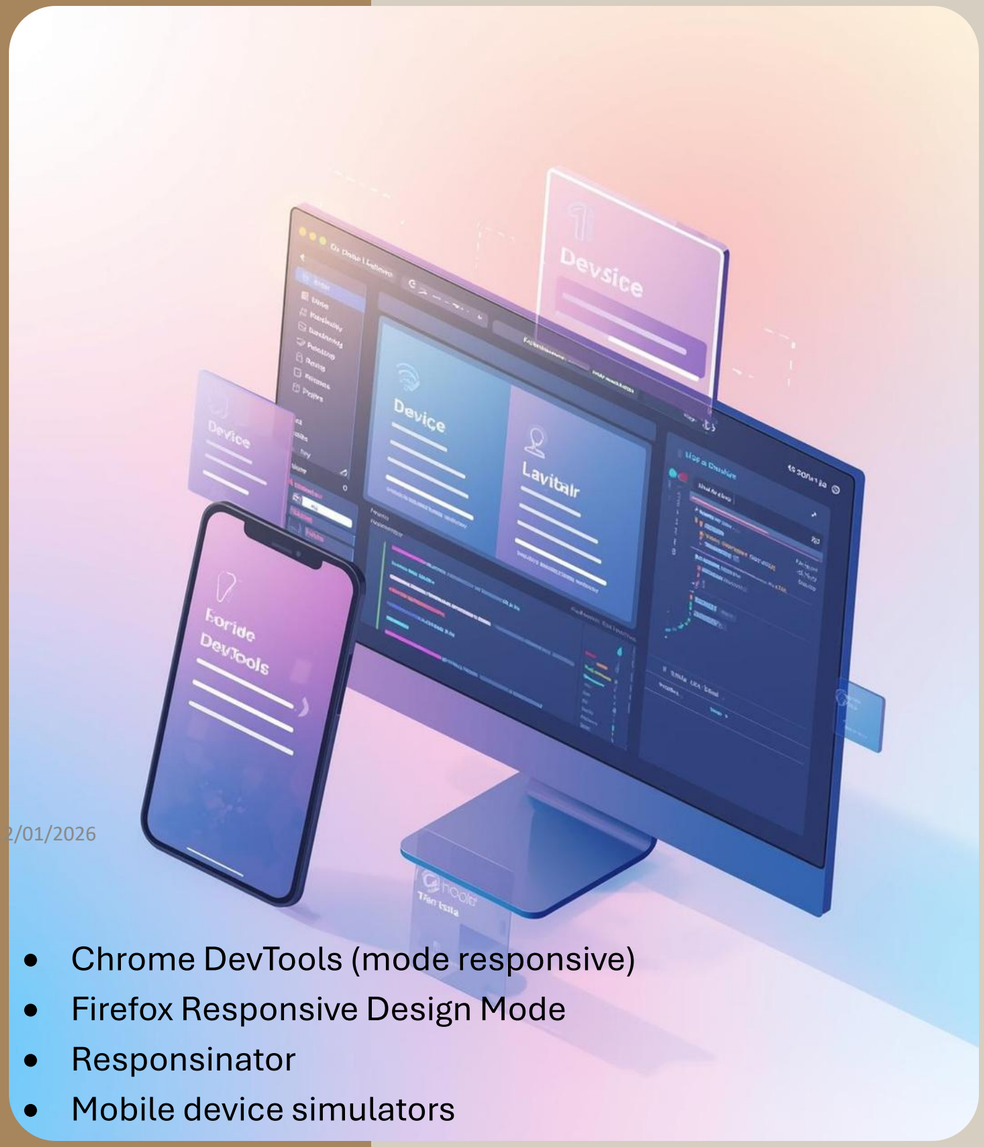
Cet icone représente une solution de navigation mobile compact



Menu horizontal

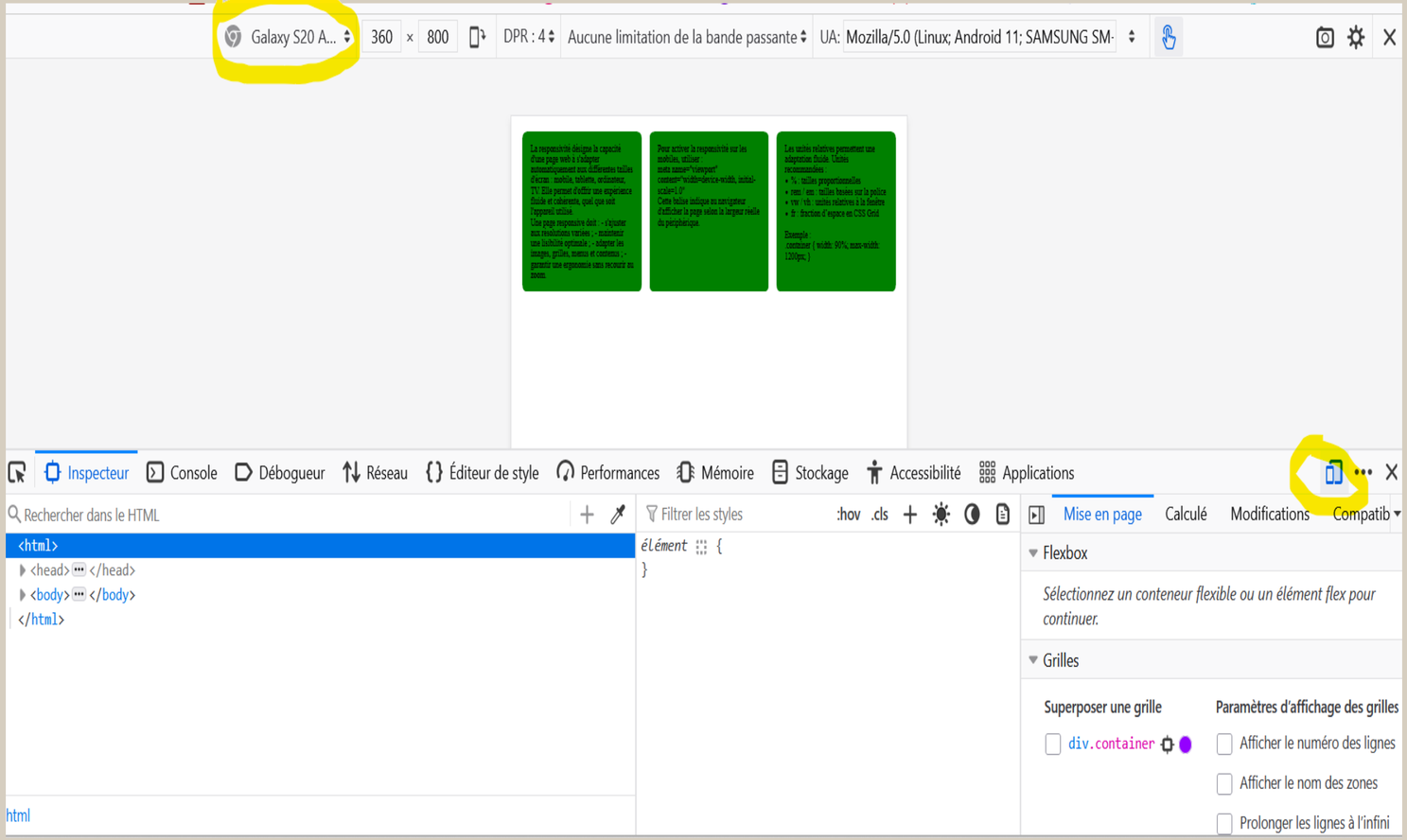
Préférable dans un desktop pour une bonne expérience de navigation

Outils de Test



- Chrome DevTools (mode responsive)
- Firefox Responsive Design Mode
- Responsinator
- Mobile device simulators

Vous pouvez aussi utiliser l'inspecteur de code



Tester sur plusieurs largeurs : - 320px (petits téléphones) - 768px (tablettes) - 1024px (laptops) - 1440px+ (desktop large)

Il permet d'afficher le site sur divers appareils, assurant une expérience utilisateur fluide et adaptée à chaque écran.

Application Pratique: Mini-Projet

12/01/2026

Structure HTML

Créez une structure de base pour votre projet web.

Grid et Flexbox

Utilisez Grid et Flexbox pour organiser les éléments.

Media Queries

Appliquez des media queries pour adapter le design.

Exemple 1 de code HTML/CSS

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <title>Page Responsive</title>
</head>
<body>

<header class="header">Mon Site Responsive</header>

<section class="grid">
  <div class="box">Bloc 1</div>
  <div class="box">Bloc 2</div>
  <div class="box">Bloc 3</div>
</section>

<footer class="footer">© Pascal</footer>

</body>
</html>
```

Exemple 1 de code HTML/CSS

```
body {
  margin: 0;
  font-family: Arial;
}
.header, .footer {
  background: #333;
  color: white;
  text-align: center;
  padding: 15px;
}
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  padding: 20px;
}
.box {
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 3px 10px rgba(0,0,0,0.1);
}
@media (max-width: 768px) {
  .grid {
    grid-template-columns: 1fr;
  }
}
```

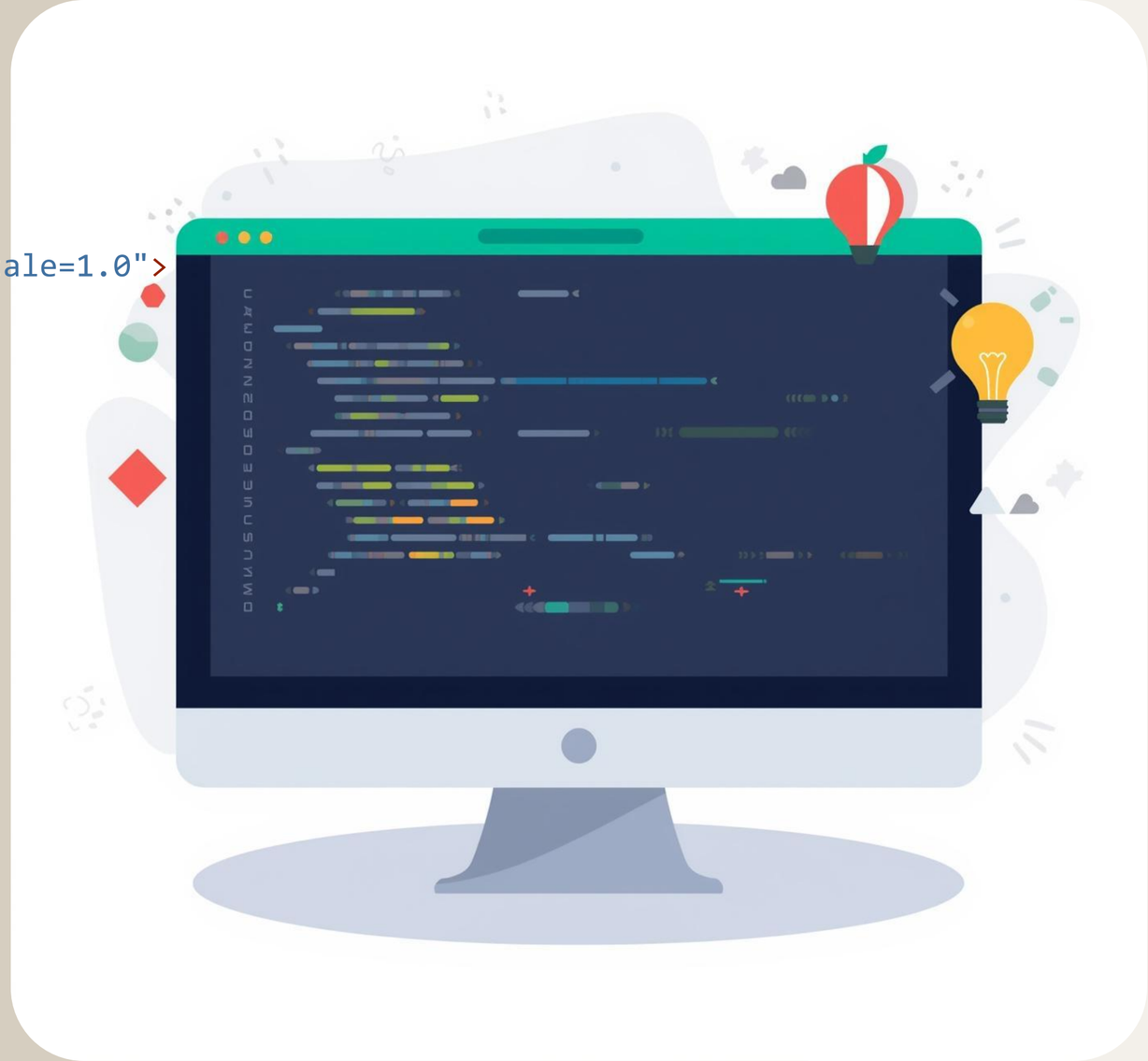
Exemple 2 de code HTML/CSS

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <title>Page Responsive</title>
</head>
<body>

<header class="header">Mon Site Responsive</header>
<nav>
  <button class="toggle">☰</button>
  <ul class="menu">
    <li>Accueil</li>
    <li>Services</li>
    <li>Contact</li>
  </ul>
</nav>

<section class="grid">
  <div class="box">Bloc 1</div>
  <div class="box">Bloc 2</div>
  <div class="box">Bloc 3</div>
</section>

<footer class="footer">© Pascal</footer>
</body>
</html>
```



Exemple 2 de code HTML/CSS

```css
body {
  margin: 0;
  font-family: Arial;
}
.header, .footer {
  background: #333;
  color: white;
  text-align: center;
  padding: 15px;
}
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  padding: 20px;
}
.box {
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 3px 10px rgba(0,0,0,0.1);
}
@media (max-width: 768px) {
  .grid {
    grid-template-columns: 1fr;
  }
}
```

```css
nav {
  background: #444;
  color: white;
  padding: 10px;
  position: relative;
}
.toggle {
  display: none;
}
.menu {
  display: flex;
  gap: 20px;
  list-style: none;
}
@media (max-width: 700px) {
  .toggle {
    display: block;
    background: none;
    border: none;
    font-size: 24px;
    color: white;
  }
  .menu {
    flex-direction: column;
    display: none;
  }
}
```

TP HTML/CSS

Objectif : créer une page responsive avec header, grille, menu responsive et footer.

Travail demandé :

1. Créer une structure HTML simple.
2. Mettre en page en utilisant Flexbox ou Grid.
3. Ajouter des media queries pour mobile (<768px).
4. Rendre les images fluides.
5. Tester la page sur simulateur responsive.

12/01/2026

TP corrigé HTML/CSS

```html
<html>
<head>
<link href="monCSS.css" rel="stylesheet" media="all" type="text/css">
</head>
<title> style</title>
<body>
<div class="container">
  <header>Mon TP</header>
  <nav>
    <button class="toggle">☰</button>
    <ul class="menu">
      <li>Accueil</li>
      <li>Blog</li>
      <li>À propos</li>
    </ul>
  </nav>
  <section class="content-grid">
    <article class="card">Article 1</article>
    <article class="card">Article 2</article>
    <article class="card">Article 3</article>
  </section>
<br/><br/><br/><br/><br/><br/><br/><br/><br/><br/><br/><br/><br/>
  <footer>© Pascal</footer>
</div>
</body>
</html>
```

TP corrigé HTML/CSS

```css
.container {
  width: 90%;
  max-width: 1200px;
  margin: auto;
}
header, footer {
  background: #333;
  color: white;
  padding: 15px;
  text-align: center;
}
nav {
  background: #555;
  padding: 10px;
  color: white;
}
.menu {
  display: flex;
  gap: 15px;
  list-style: none;
}
.toggle { display: none; }
```

```css
.content-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  padding: 20px 0;
}
.card {
  background: white;
  padding: 20px;
  border-radius: 10px;
}
@media (max-width: 768px) {
  .content-grid {
    grid-template-columns: 1fr;
  }
  .toggle {
    display: block;
    background: none;
    border: none;
    font-size: 24px;
    color: white;
  }
  .menu {
    flex-direction: column;
    display: none;
  }
}
```

“Un site responsive, c’est un site qui s’adapte à l’utilisateur, pas l’inverse.”

12/01/2026